

[HCI/2024/Q3]

3 Name your Jupyter Notebook as

TASK3_<your name>_<centre number>_<index number>.ipynb

The task is to implement a task scheduler application using a linked list to store tasks in a priority queue to manage the execution order.

Tasks should execute in order of their priority, with higher priority tasks taking precedence. Additionally, tasks with the same priority should be executed in a first-come-first-serve manner.

The class `TaskNode` contains three attributes:

- `taskName`: Unique task name.
- `priority`: An integer priority of the task, with a higher number indicating higher priority.
- `next`: Reference to the next task node in the linked list.

The class `TaskNode` has the following method:

- `__str__()`: Returns a string representation of the task details (`taskName`, `priority`).

The class `PriorityQueue` contains one attribute:

- `head`: Reference to the first node in the queue.

The class `PriorityQueue` contains the following methods:

- `viewPendingTasks()`: Displays all tasks that are pending.
- `enqueue(taskNode)`: Adds a task node to the queue in the correct position based on its priority.
- `dequeue()`: Removes and returns the task node from the front of the queue. If the queue is empty, returns `None`.
- `isEmpty()`: Returns `True` if the queue is empty, or `False` otherwise.
- `delete(taskName)`: Deletes a task with the given `taskName`. Returns `True` if successful, or `False` otherwise.
- `search(taskName)`: Searches for a task node with the given `taskName` and returns it if found; else returns `None`.
- `updatePriority(taskName, newPriority)`: Updates the priority of a task with the given `taskName` and reorders the queue if necessary.

[HCI/2024/Q3 Cont']

For each of the sub-tasks, add a comment statement at the beginning of the code using the hash symbol '#', to indicate the sub-task the program code belongs to, for example:

In [1]:

```
#Task 3.1  
Program Code
```

Output:

Task 3.1

Write the program code for the `TaskNode` class.

[2]

Task 3.2

Write the program code for the `PriorityQueue` class, including the following methods:

- `viewPendingTasks()`
- `enqueue(taskNode)`
- `dequeue()`
- `isEmpty()`

[10]

A text file named `TASKS.txt` containing task data in the following format:
`taskName, priority`

In a new cell, write the program code to:

- create a `PriorityQueue` object
- enqueue all tasks read from `TASKS.txt`
- display all pending tasks
- dequeue **two** tasks and display the remaining pending tasks.

Test your program and show the output. All outputs should have appropriate messages to indicate what they are showing.

[4]

[HCI/2024/Q3 Cont']

Task 3.3

Amend your code for **Task 3.2** to include the following methods in the `PriorityQueue` class:

- `delete(taskName)`
- `search(taskName)`
- `updatePriority(taskName, newPriority)` [10]

In a new cell, write the program code to:

- search for 'Task 1' and print the result
- delete 'Task 5' and display the updated queue
- update the priority of 'Task 4' to 4 and display the updated pending tasks.

Test your program and show the output. All outputs should have appropriate messages to indicate what they are showing. [3]

Save your Jupyter Notebook for Task 3.